

Karaoke CT

Documentacao Completa doCodigo

Guia comentado para desenvolvedores - PT/BR

Plataforma profissional de karaoke com catalogo de ~80.000 musicas, player integrado com Bunny Stream, painel admin e sistema de cupons de acesso temporario com captura de leads.

SUMARIO

• 1. Visao Geral da Arquitetura	3
• 2. Banco de Dados - Schema	5
• 3. API Server - Rotas	8
• 4. Frontend - Contextos	15
• 5. Frontend - Paginas	22
• 6. Frontend - Componentes	30
• 7. OpenAPI Spec	35
• 8. Guia de Modificacoes	38

1. VISAO GERAL DA ARQUITETURA

O Karaoke CT e uma aplicacao fullstack dividida em **3 camadas principais**:

1. Banco de Dados (PostgreSQL + Drizzle ORM)

Armazena musicas, assinantes e cupons de acesso. Localizado em *lib/db/src/schema/*.

2. API Server (Express 5 + TypeScript)

Serve os endpoints REST. Localizado em *artifacts/api-server/src/routes/*.

3. Frontend (React + Vite + Tailwind + shadcn/ui)

Interface do usuario. Localizado em *artifacts/karaoke/src/*.

Fluxo de Dados:

O frontend faz chamadas HTTP para a API. A API consulta o banco via Drizzle ORM. O OpenAPI spec (*lib/api-spec/openapi.yaml*) define todos os contratos e gera automaticamente os hooks React Query via Orval.

2. BANCO DE DADOS - SCHEMA

2.1. Tabela de Musicas (musicas.ts)

O que faz: Define a estrutura da tabela que armazena todas as musicas do catalogo.

Campos: id (numerico, PK), artista, musica (titulo), inicio (frase inicial opcional).

Importante: O ID deve ser igual ao ID do video no Bunny Stream para o player funcionar.

```
import { pgTable, integer, text } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod/v4";

// Cria a tabela 'musicas' no PostgreSQL
export const musicasTable = pgTable("musicas", {
  id: integer("id").primaryKey(), // ID = ID do video no Bunny Stream
  artista: text("artista").notNull(), // Nome do artista/banda
  musica: text("musica").notNull(), // Titulo da musica
  inicio: text("inicio"), // Frase de inicio (ex: Tem um pedaco...)
});

// Schema Zod para validacao de insercao
export const insertMusicaSchema = createInsertSchema(musicasTable);
export type InsertMusica = z.infer<typeof insertMusicaSchema>;
export type Musica = typeof musicasTable.$inferSelect;
```

Como modificar: Se quiser adicionar um campo novo (ex: genero, ano), adicione aqui e depois rode *pnpm --filter @workspace/db run push*.

2.2. Tabela de Assinantes (users.ts)

O que faz: Armazena os usuarios com assinatura ativa. Integra com WooCommerce via webhook.

Campos principais: email, password_hash, subscription_status, access_granted, active_session_token.

Seguranca: A senha e armazenada como hash (bcryptjs). Apenas 1 sessao ativa por vez.

```
import { pgTable, serial, text, boolean, timestamp } from "drizzle-orm/pg-core";
import { createInsertSchema } from "drizzle-zod";
import { z } from "zod/v4";

export const karaokeUsersTable = pgTable("karaoke_users", {
  id: serial("id").primaryKey(),
  email: text("email").notNull().unique(), // Email do assinante
  passwordHash: text("password_hash"), // Hash da senha (bcryptjs)
  woocommerceCustomerId: text("woocommerce_customer_id"),
  subscriptionStatus: text("subscription_status").notNull().default("active"),
  accessGranted: boolean("access_granted").notNull().default(true),
  accessGrantedAt: timestamp("access_granted_at", { withTimezone: true }).defaultNow(),
  expiresAt: timestamp("expires_at", { withTimezone: true }),
  activeSessionToken: text("active_session_token"), // Token unico da sessao
  activeSessionAt: timestamp("active_session_at", { withTimezone: true }),
});
```

Como funciona a sessao unica: Quando o usuario faz login, um token aleatorio e gerado e salvo. Se outro dispositivo logar, o token antigo e sobrescrito e o primeiro dispositivo perde o acesso.

2.3. Tabela de Cupons (access-codes.ts)

O que faz: Armazena os codigos de acesso temporario para marketing/eventos.

Novidade: Campos redeemerName, redeemerEmail, redeemerWhatsapp capturam dados do lead no momento do resgate.

```
export const accessCodesTable = pgTable("access_codes", {
  id: serial("id").primaryKey(),
  code: varchar("code", { length: 16 }).notNull().unique(), //Codigo de 8 chars
  durationMinutes: integer("duration_minutes").notNull(), // Duracao do acesso
  label: text("label"), // Identificacao (ex: Black Friday)
  used: boolean("used").notNull().default(false), // Ja foi usado?
  usedAt: timestamp("used_at", { withTimezone: true }),
  usedBy: text("used_by"), // Email de quem resgatou
  redeemerName: text("redeemer_name"), // NOME do lead (novo)
  redeemerEmail: text("redeemer_email"), // EMAIL do lead (novo)
  redeemerWhatsapp: text("redeemer_whatsapp"), // WHATSAPP do lead (novo)
  expiresAt: timestamp("expires_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow(),
  createdBy: integer("created_by"), // ID do admin que criou
}):
```

Para que servem os novos campos: Quando alguém resgata um cupom na tela de login, esses 3 campos salvam os dados de contato. No painel admin (aba Cupons), você vê esses dados para campanhas de marketing.

3. API SERVER - ROTAS

3.1. Rota de Cupons (access-codes.ts)

Arquivo: artifacts/api-server/src/routes/access-codes.ts

Endpoints: POST /access-codes (gerar), GET /access-codes (listar), POST /access-codes/:code/redeem (resgatar), GET /access-codes/validate (validar).

Logica: Cada codigo e unico, gerado com crypto.randomBytes. Ao resgatar, marca como 'used', salva os dados do lead e calcula a data de expiracao.

```
// GERA UM CODIGO ALEATORIO DE 8 CARACTERES (ex: 7E0C7D1A)
function generateCode(): string {
  return randomBytes(4).toString("hex").toUpperCase();
}

// ADICIONA MINUTOS A UMA DATA
function addMinutes(date: Date, minutes: number): Date {
  return new Date(date.getTime() + minutes * 60_000);
}
```

Explicacao: randomBytes(4) gera 4 bytes aleatorios (32 bits). toString('hex') converte para hexadecimal (8 caracteres). Isso da ~4 bilhoes de combinacoes possiveis, mais que suficiente.

Gerar cupons (POST /access-codes):

Recebe: durationMinutes, quantity, label. Faz: Gera codigos unicos, evita duplicatas (loop com checagem no DB), salva no banco. Retorna: Lista de codigos gerados com suas datas de expiracao.

```
router.post("/access-codes", async (req, res) => {
  const { durationMinutes, quantity, label } = req.body ?? {};
  if (!durationMinutes || durationMinutes <= 0) {
    res.status(400).json({ error: "durationMinutes invalido" }); return;
  }
  const qty = typeof quantity === "number" ? Math.min(quantity, 50) : 1;
  const results = [];
  for (let i = 0; i < qty; i++) {
    let code, unique = false, attempts = 0;
    do {
      code = generateCode();
      const existing = await db.select({ id: accessCodesTable.id })
        .from(accessCodesTable).where(eq(accessCodesTable.code, code));
      unique = existing.length === 0; attempts++;
    } while (!unique && attempts < 10);
    const expiresAt = addMinutes(new Date(), durationMinutes);
    const [record] = await db.insert(accessCodesTable).values({ code, durationMinutes, label, expiresAt }).returning();
    results.push({ code, durationMinutes, label, expiresAt });
  }
  res.status(201).json({ success: true, codes: results });
});
```

Resgatar cupom (POST /access-codes/:code/redeem):

Esse e o endpoint mais importante para captura de leads. Recebe: name, email, whatsapp no body. Valida: Codigo existe? Nao expirou? Nao foi usado? Salva: Marca como 'used' e grava os dados do lead nos campos redeemerName, redeemerEmail, redeemerWhatsapp.

```
router.post("/access-codes/:code/redeem", async (req, res) => {
  const rawCode = req.params.code?.trim().toUpperCase();
  if (!rawCode) { res.status(400).json({ error: "Codigo obrigatorio" }); return; }
  const [code] = await db.select().from(accessCodesTable).where(eq(accessCodesTable.code, rawCode));
```

```
if (!code) { res.status(404).json({ error: "Codigo nao encontrado" }); return; }
if (code.expiresAt && code.expiresAt < new Date()) { res.status(410).json({ error: "Codigo expirado" }); return; }
if (code.used) { res.status(409).json({ error: "Codigo ja utilizado" }); return; }
// CAPTURA DOS DADOS DO LEAD
const usedBy = req.body?.email?.trim().toLowerCase();
const redeemerName = req.body?.name?.trim();
const redeemerEmail = usedBy;
const redeemerWhatsapp = req.body?.whatsapp?.trim();
const accessExpiresAt = addMinutes(new Date(), code.durationMinutes);
await db.update(accessCodesTable).set({
  used: true, usedAt: new Date(), usedBy,
  redeemerName, redeemerEmail, redeemerWhatsapp,
}).where(eq(accessCodesTable.id, code.id));
res.json({ success: true, message: `Acesso liberado por ${code.durationMinutes} minutos!`,
  durationMinutes: code.durationMinutes, accessExpiresAt: accessExpiresAt.toISOString() });
});
```

Onde mudar se quiser mais campos: Adicione novos campos no schema

(lib/db/src/schema/access-codes.ts), depois inclua no .set() acima e no frontend (login.tsx).

3.2. Rota de Usuarios (users.ts)

Arquivo: artifacts/api-server/src/routes/users.ts

Endpoints: GET /users, POST /users/login, POST /users/logout, GET /users/me, POST /users/:id/password, POST /users/:id/revoke.

Seguranca: Senhas com bcryptjs, tokens de sessao de 32 bytes, sessao unica por dispositivo.

```
// LOGIN: Verifica email e senha, gera token de sessao
router.post("/users/login", async (req, res) => {
  const email = req.body?.email?.trim().toLowerCase();
  const password = req.body?.password?.trim();
  const [user] = await db.select().from(karaokeUsersTable).where(eq(karaokeUsersTable.email, email));
  if (!user) { res.status(401).json({ error: "Email ou senha incorretos." }); return; }
  if (!user.accessGranted) { res.status(403).json({ error: "Acesso revogado." }); return; }
  const valid = await bcryptjs.compare(password, user.passwordHash);
  if (!valid) { res.status(401).json({ error: "Email ou senha incorretos." }); return; }
  // Gera novo token e invalida sessoes antigas (sessao unica)
  const token = randomBytes(32).toString("hex");
  await db.update(karaokeUsersTable).set({ activeSessionToken: token, activeSessionAt: new Date() }).where(eq(karaokeUsersTable.id, user.id));
  res.json({ token, id: user.id, email: user.email, ... });
});
```

Como a sessao unica funciona: O token e salvo no banco. O frontend envia esse token no header 'X-Auth-Token' em todas as requisicoes. Se outro dispositivo logar, o token antigo e sobrescrito e o primeiro fica sem acesso.

3.3. Rota de Musicas (musicas.ts)

Arquivo: artifacts/api-server/src/routes/musicas.ts

Endpoints: GET /musicas/search (busca com paginacao), GET /musicas/stats, POST /musicas (cadastrar), PUT /musicas/:id, DELETE /musicas/:id.

Busca: ILIKE nas colunas artista, musica e inicio. Se o termo for so numeros, busca por ID exato.

```
router.get("/musicas/search", async (req, res) => {
  const { q, page = 1, limit = 24 } = req.query;
  const offset = (page - 1) * limit;
  const isNumeric = /^\d+$/.test(q);
  const numericId = isNumeric ? parseInt(q) : null;
  const where = q
    ? isNumeric
      ? eq(musicasTable.id, numericId)
      : or(
        ilike(musicasTable.musica, `_${q}_`),
        ilike(musicasTable.artista, `_${q}_`),
        ilike(musicasTable.inicio, `_${q}_`),
      )
    : undefined;
  const [data, [totalRow]] = await Promise.all([
    db.select().from(musicasTable).where(where).orderBy(musicasTable.artista).limit(limit).offset(
      offset),
    db.select({ count: count() }).from(musicasTable).where(where),
  ]);
  res.json({ data, total: Number(totalRow.count), page, limit, totalPages:
    Math.ceil(Number(totalRow.count) / limit) });
});
```


4. FRONTEND - CONTEXTOS REACT

4.1. Contexto de Acesso Temporario (temporary-access-context.tsx)

Arquivo: artifacts/karaoke/src/contexts/temporary-access-context.tsx

Funcao: Gerencia o estado do acesso via cupom no frontend. Salva no localStorage para persistir entre recarregamentos.

Features: Timer que conta os minutos restantes, expiracao automatica, funcao redeemCode que chama a API.

```
interface TemporaryAccess {
  code: string; // Codigo do cupom
  durationMinutes: number; // Quantos minutos de acesso
  accessExpiresAt: string; // Data/hora de expiracao (ISO)
  redeemedAt: string; // Quando foi resgatado
}

// CHAVE usada no localStorage para persistir o acesso
const STORAGE_KEY = "karaoke-ct-temp-access";

// Carrega do localStorage e verifica se ja expirou
function loadAccess(): TemporaryAccess | null {
  const raw = localStorage.getItem(STORAGE_KEY);
  if (!raw) return null;
  const parsed = JSON.parse(raw);
  if (new Date(parsed.accessExpiresAt) < new Date()) {
    localStorage.removeItem(STORAGE_KEY); // Limpa se expirou
    return null;
  }
  return parsed;
}

// Funcao que resgata o cupom chamando a API
const redeemCode = useCallback(async (code, name, email, whatsapp) => {
  const res = await fetch(`/api/access-codes/${code}/redeem`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, email: email.toLowerCase(), whatsapp }),
  });
  if (!res.ok) return { success: false, error: "Erro ao resgatar" };
  const data = await res.json();
  const newAccess = {
    code: code.toUpperCase(),
    durationMinutes: data.durationMinutes,
    accessExpiresAt: data.accessExpiresAt,
    redeemedAt: new Date().toISOString(),
  };
  localStorage.setItem(STORAGE_KEY, JSON.stringify(newAccess));
  setAccess(newAccess);
  return { success: true, message: data.message };
}, []);
```

Timer de expiracao: Um useEffect roda a cada 30 segundos, recalcula os minutos restantes e limpa o acesso se chegar a zero. Isso garante que o usuario nao fique com acesso alem do tempo comprado.

4.2. Contexto de Autenticacao (auth-context.tsx)

Arquivo: artifacts/karaoke/src/context/auth-context.tsx

Funcao: Gerencia login/senha dos assinantes. Similar ao temporary-access, mas usa token de sessao da API.

Fluxo: Login -> recebe token -> salva no localStorage -> valida token ao recarregar a pagina -> logout limpa tudo.

```
const AUTH_KEY = "karaoke-ct-auth-token";

// LOGIN: Envia email/senha, recebe token, salva no localStorage
const login = useCallback(async (email, password) => {
  const res = await fetch("/api/users/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email: email.trim().toLowerCase(), password }),
  });
  if (!res.ok) { setError("Erro ao fazer login"); return false; }
  const data = await res.json();
  localStorage.setItem(AUTH_KEY, data.token);
  setUser({ id: data.id, email: data.email, ... });
  return true;
}, []);

// Ao iniciar a app, valida o token salvo
useEffect(() => {
  const token = localStorage.getItem(AUTH_KEY);
  if (token) {
    fetch("/api/users/me", { headers: { "X-Auth-Token": token } })
      .then(res => res.ok ? res.json() : null)
      .then(user => { if (user) setUser(user); else localStorage.removeItem(AUTH_KEY); });
  }
}, []);
```

5. FRONTEND - PAGINAS

5.1. Login (login.tsx)

Arquivo: artifacts/karaoke/src/pages/login.tsx

O que faz: Tela de login com duas opcoes: entrar com email/senha (assinantes) ou usar cupom de acesso temporario (leads).

Parte nova: Formulario de captura de leads com Nome, Email, WhatsApp e Codigo do cupom.

```
// Estados do formulario de login
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");

// Estados do formulario de cupom (NOVO - captura de leads)
const [cName, setCName] = useState(""); // Nome do lead
const [cEmail, setCEmail] = useState(""); // Email do lead
const [cWhatsapp, setCWhatsapp] = useState(""); // WhatsApp do lead
const [cCode, setCCode] = useState(""); // Codigo do cupom
const [showCoupon, setShowCoupon] = useState(false);

// Se ja esta logado ou tem acesso temporario, redireciona para home
if (user || hasAccess) { navigate("/"); return null; }

// FUNCAO QUE RESGATA O CUPOM
async function handleRedeem() {
  if (!cName.trim() || !cEmail.trim() || !cWhatsapp.trim() || !cCode.trim()) {
    toast({ title: "Preencha todos os campos", variant: "destructive" });
    return;
  }
  setRedeeming(true);
  const result = await redeemCode(cCode.trim(), cName.trim(), cEmail.trim(), cWhatsapp.trim());
  setRedeeming(false);
  if (result.success) {
    toast({ title: "Cupom ativado!", description: result.message });
    navigate("/");
  } else {
    toast({ title: "Erro ao ativar cupom", description: result.error, variant: "destructive" });
  }
}
```

Como modificar os campos: Se quiser adicionar 'CPF' ou 'Cidade', crie um novo useState, adicione o input no JSX, e passe como parametro no redeemCode(). Depois atualize a API e o schema do banco.

5.2. Admin (admin.tsx)

Arquivo: artifacts/karaoke/src/pages/admin.tsx

O que faz: Painel administrativo com 3 abas: Catalogo (cadastrar musicas), Assinantes (gerenciar acesso), Cupons (gerar e visualizar codigos).

Aba Cupons: Importa o componente CuponsTab separado.

```
<Tabs defaultValue="catalog">
  <TabList>
    <TabTrigger value="catalog">Catalogo</TabTrigger>
    <TabTrigger value="subscribers">Assinantes</TabTrigger>
    <TabTrigger value="cupons">Cupons</TabTrigger>
  </TabList>
  <TabContent value="catalog">
    <Form>...cadastrar nova musica...</Form>
  </TabContent>
  <TabContent value="subscribers">
    <Table>...lista de assinantes...</Table>
  </TabContent>
  <TabContent value="cupons">
```

```
<CouponsTab />  
</TabsContent>  
</Tabs>
```

6. FRONTEND - COMPONENTE CUPONS

6.1. CuponsTab (cupons-tab.tsx)

Arquivo: artifacts/karaoke/src/components/cupons-tab.tsx

O que faz: Componente completo da aba Cupons no admin. Tem duas partes: gerador de cupons e lista com dados dos leads.

Hooks usados: useCreateAccessCodes (gerar), useListAccessCodes (listar) - gerados automaticamente pelo Orval.

```
const DURATIONS = [
  { value: "15", label: "15 minutos" },
  { value: "30", label: "30 minutos" },
  { value: "60", label: "1 hora" },
  { value: "240", label: "4 horas" },
  { value: "360", label: "6 horas" },
  { value: "720", label: "12 horas" },
  { value: "1440", label: "24 horas" },
];

// FUNCAO PARA GERAR CUPONS
function onGenerate() {
  const mins = parseInt(duration, 10);
  createCodes.mutate({ data: { durationMinutes: mins, quantity, label } }, {
    onSuccess: (result) => {
      toast({ title: "Cupons gerados!", description: `${result.codes.length} codigo(s) criados.` });
      queryClient.invalidateQueries({ queryKey: ["/api/access-codes"] });
    },
  });
}

// COLUNAS DA TABELA (com dados dos leads)
<TableHeader>
  <TableRow>
    <TableHead>Codigo</TableHead>
    <TableHead>Status</TableHead>
    <TableHead>Duracao</TableHead>
    <TableHead>Identificacao</TableHead>
    <TableHead>Nome</TableHead> <!-- NOVO: lead name -->
    <TableHead>Email</TableHead> <!-- NOVO: lead email -->
    <TableHead>WhatsApp</TableHead> <!-- NOVO: lead whatsapp -->
    <TableHead>Criado em</TableHead>
    <TableHead>Acoes</TableHead>
  </TableRow>
</TableHeader>
```

Status visual: 'Pendente' = verde, 'Usado' = vermelho, 'Expirado' = cinza. Isso ajuda o admin a identificar rapidamente quais codigos ainda podem ser distribuidos.

7. OPENAPI SPEC (openapi.yaml)

Arquivo: lib/api-spec/openapi.yaml

Funcao: Define TODOS os contratos da API em uma linguagem universal. A partir deste arquivo, o comando `pnpm --filter @workspace/api-spec run codegen` gera automaticamente:

- Schemas Zod (validacao)
- Hooks React Query (para o frontend usar)
- Tipos TypeScript

Regra: NUNCA mude o 'info.title' - ele define os nomes dos arquivos gerados.

```
openapi: 3.1.0
info:
  title: Api # NAO MUDE! Define os nomes dos arquivos gerados
  version: 0.1.0
servers:
  - url: /api
paths:
  /access-codes:
    get:
      operationId: listAccessCodes # Hook: useListAccessCodes
    post:
      operationId: createAccessCodes # Hook: useCreateAccessCodes
  /access-codes/{code}/redeem:
    post:
      operationId: redeemAccessCode
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [name, email, whatsapp] # CAMPOS OBRIGATORIOS
              properties:
                name: { type: string }
                email: { type: string }
                whatsapp: { type: string }
```

Workflow: 1) Edite o openapi.yaml -> 2) Rode codegen -> 3) Os hooks aparecem automaticamente em @workspace/api-client-react. Se mudar o 'operationId', o nome do hook muda tambem.

8. APP.TSX - MONTAGEM DA APLICACAO

Arquivo: artifacts/karaoke/src/App.tsx

Funcao: E o ponto de entrada do React. Empilha todos os providers (contextos) e define as rotas. **Ordem dos providers importa:** Os que estao por fora sao acessiveis por todos os de dentro.

```
<ThemeProvider defaultTheme="dark"> <!-- Tema escuro por padrao -->
<QueryClientProvider client={queryClient}> <!-- React Query para cache de API -->
<TooltipProvider> <!-- Tooltips do shadcn -->
<LocalMusicProvider> <!-- Musicas locais -->
<QueueProvider> <!-- Fila de musicas -->
<SessionProvider> <!-- Sessao TV/Remoto -->
<AuthProvider> <!-- Login com senha -->
<TemporaryAccessProvider> <!-- Acesso via cupom -->
<SearchProvider> <!-- Estado da busca -->
<WouterRouter> <!-- Roteamento -->
<Switch>
<Route path="/" component={Home} />
<Route path="/player/:id" component={Player} />
<Route path="/admin" component={Admin} />
<Route path="/tv/:sessionId" component={TVPage} />
<Route path="/remote/:sessionId" component={RemotePage} />
<Route path="/login" component={LoginPage} />
</Switch>
</WouterRouter>
<Toaster />
</SearchProvider>
</TemporaryAccessProvider>
</AuthProvider>
</SessionProvider>
</QueueProvider>
</LocalMusicProvider>
</TooltipProvider>
</QueryClientProvider>
</ThemeProvider>
```

Como adicionar uma nova pagina: 1) Crie o componente em pages/ -> 2) Importe em App.tsx -> 3) Adicione uma Route com o path e componente.

9. GUIA DE MODIFICACOES COMUNS

A) Adicionar um campo novo no formulario de cupom (ex: CPF)

1. lib/db/src/schema/access-codes.ts - Adicione o campo
2. pnpm --filter @workspace/db run push - Aplica no banco
3. lib/api-spec/openapi.yaml - Adicione no requestBody
4. pnpm --filter @workspace/api-spec run codegen - Regenera hooks
5. artifacts/api-server/src/routes/access-codes.ts - Capture e salve
6. artifacts/karaoke/src/pages/login.tsx - Adicione input
7. artifacts/karaoke/src/components/cupons-tab.tsx - Adicione coluna

B) Mudar o tempo padrao dos cupons

Edite o array DURATIONS em cupons-tab.tsx.

C) Mudar as cores do tema

Edite as variaveis CSS em artifacts/karaoke/src/index.css.

D) Adicionar uma nova rota na API

1. Defina no openapi.yaml -> 2. Rode codegen -> 3. Crie em routes/ -> 4. Importe no servidor.

10. COMANDOS UTEIS

Desenvolvimento:

- `pnpm --filter @workspace/api-server run dev` - Inicia o servidor API
- `pnpm --filter @workspace/karaoke run dev` - Inicia o frontend
- `pnpm run typecheck` - Verifica tipos em todo o projeto

Banco de Dados:

- `pnpm --filter @workspace/db run push` - Aplica mudancas do schema
- `pnpm --filter @workspace/db run studio` - Abre o Drizzle Studio

API / Codegen:

- `pnpm --filter @workspace/api-spec run codegen` - Regenera hooks do OpenAPI

Importacao:

- `pnpm --filter @workspace/scripts run import-musicas -- --file /caminho/Dados_CT.ini` - Importa catalogo

Documentacao gerada em 15 de Maio de 2026. Para duvidas, consulte o [replit.md](#) na raiz do projeto.